SIMATS ENGINEERING

CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 3 – CO3

Full Stack Development and Containerization of the Smart Campus Facility Platform

Student Details	Information
Student Name	Doma Saketh
Register Number	192525018
Programme	B.Tech – Artificial Intelligence and Machine Learning
Faculty Name	_____
Assignment Type	Individual
Duration	One week
Total Marks	50
Submission Date	_____

CO3: Develop full-stack applications using version control systems like Git/GitHub and containerization tools like Docker and Kubernetes.

Declaration: This submission contains a runnable source project and design evidence. Runtime, GitHub history, Docker deployment and concurrency results must be generated from the student's own environment before final submission; no unperformed execution is represented as completed.

Executive Summary

CampusOps is a full-stack academic prototype for facility booking and maintenance operations. It supports a facility catalogue, booking requests, overlap prevention, approval-controlled booking transitions, maintenance tickets, technician workflow, sensor-event intake, threshold-based ticket generation and audit events. The selected stack follows the assignment: Vue 3 + TypeScript, Django REST Framework, PostgreSQL, Git/GitHub and Docker Compose.

System Scope and Assumptions

Area	Definition
Users	Administrator, facility manager, maintenance technician, department approver, faculty member and student.
Facilities	Laboratories, seminar halls, workshop rooms and shared equipment spaces.
Booking	Requested, Approved, Confirmed, In Use, Completed and Cancelled states.
Maintenance	Open, Assigned, In Progress, Resolved and Closed states.
Sensor input	Controlled sensor events; temperature >=45 triggers a high-priority maintenance ticket.
Constraints	One-week assignment scope; external issue tracker/calendar may be unavailable; preserve local records and retry safely.
Scale challenge	Six campuses, 300 peak users, overlapping reservations, conflicting updates and one backend instance restarting.

Q1 – Define Application Scope and Select Technology Stack (5 Marks)

1.1 Principal Users and Workflows

User	Workflow	Access boundary
Student / Faculty	Browse facilities, request booking, view own booking and report issue.	Own records; no approval or technician actions.
Facility manager	Review booking queue, approve/reject, view facility schedule and reports.	Facility-wide booking view and approval.
Department approver	Approve restricted facility or high-impact booking.	Approval permission within assigned scope.
Maintenance technician	View assigned tickets, update status, add resolution note.	Assigned and operational tickets only.
Administrator	Manage facilities, users, roles and audit.	Privileged configuration and audit access.

1.2 User Stories and Acceptance Criteria

ID	User story	Measurable acceptance criterion
US01	As a student, I want to browse active facilities so I can select a suitable space.	Catalogue shows active facility name, building and capacity; inactive facilities are not bookable.
US02	As a student, I want to request a time slot so I can reserve a lab.	Valid request creates REQUESTED record and remains after page refresh.
US03	As a facility manager, I want overlapping bookings rejected.	Second overlapping request returns validation error and the first booking remains unchanged.
US04	As a department approver, I want restricted bookings approved by an authorized user.	Authorized approver succeeds; student receives HTTP 403; decision is audited.
US05	As a technician, I want maintenance tickets assigned and resolved.	Ticket moves OPEN → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED; invalid transitions are rejected.
US06	As a facility manager, I want sensor alerts to create maintenance tickets.	A unique temperature event >=45 creates one HIGH ticket; duplicate event ID is rejected.

1.3 Technology Comparison

Layer	Selected option	Alternative considered	Reason / trade-off
Frontend	Vue 3 + TypeScript + Vite	Angular / React	Lightweight component framework and type checking; requires separate API client.
Backend	Django REST Framework	Spring Boot	Fast CRUD, serializers, ORM and permission framework; Python runtime.
Database	PostgreSQL	MySQL	Transactions, constraints, JSON support and row locking; needs DB operations.
Version control	Git + GitHub	Centralized SVN	Branching, pull requests, review and tagged releases.
Containerization	Docker Compose	Kubernetes	Reproducible multi-container local release; Kubernetes is unnecessary for this one-week scope.

Selected stack: Vue, Django REST Framework and PostgreSQL. The selection prioritizes fast implementation, server-side validation and persistent relational records. Kubernetes is not selected because the assignment's first release does not justify cluster orchestration.

1.4 Success Criteria

A student can submit a valid booking and retrieve it after refresh.

Overlapping bookings are rejected by the backend, not merely hidden in the UI.

Unauthorized users cannot approve bookings or change technician ticket states.

Ticket and sensor records persist in PostgreSQL.

A clean deployment can be rebuilt from committed source and configuration without committing secrets.

Q2 – Create the Integrated Application Design (7 Marks)

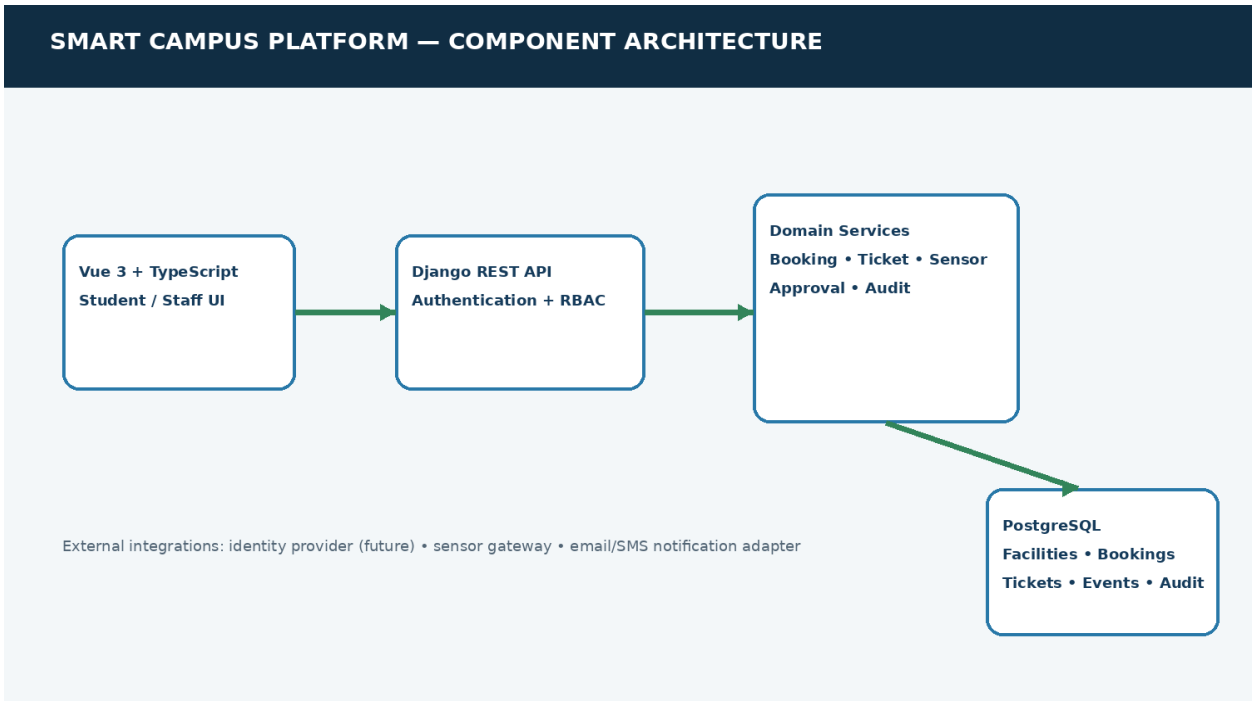


Figure 1. Component architecture: Vue UI → Django REST API → domain services → PostgreSQL.

2.1 Domain Data Model

Entity	Important fields	Relationship / purpose
User	id, username, role/staff status	Requester, approver, reporter, assignee and audit actor.
Facility	id, name, building, capacity, equipment, active	One facility has many bookings, tickets and sensor events.
Booking	id, facility, requester, purpose, start_at, end_at, status, version	Facility reservation and state history through audit records.
MaintenanceTicket	id, facility, reporter, assignee, priority, status, SLA due	Issue lifecycle and technician assignment.
SensorEvent	event_id unique, facility, metric, value, observed_at	Idempotent event intake and threshold processing.
Approval	booking/ticket, approver, decision, reason, created_at	Accountable decision record.
AuditEvent	actor, entity_type, entity_id, action, detail, created_at	Traceable state change evidence.

2.2 API Contracts

Endpoint	Method	Purpose / rule
/api/health/	GET	Public liveness response.
/api/facilities/	GET	Active facility catalogue.
/api/bookings/	GET / POST	List permitted bookings; create request with facility, purpose, start and end.
/api/bookings/{id}/transition/	POST	Approve, confirm, start, complete or cancel using guarded transition.
/api/tickets/	GET / POST	List scoped tickets or report a maintenance issue.
/api/tickets/{id}/transition/	POST	Technician/manager status changes.
/api/sensor-events/	GET / POST	Technician/manager sensor event intake; unique event ID.
/api/audit/	GET	Manager/admin audit view.

2.3 Booking State Machine

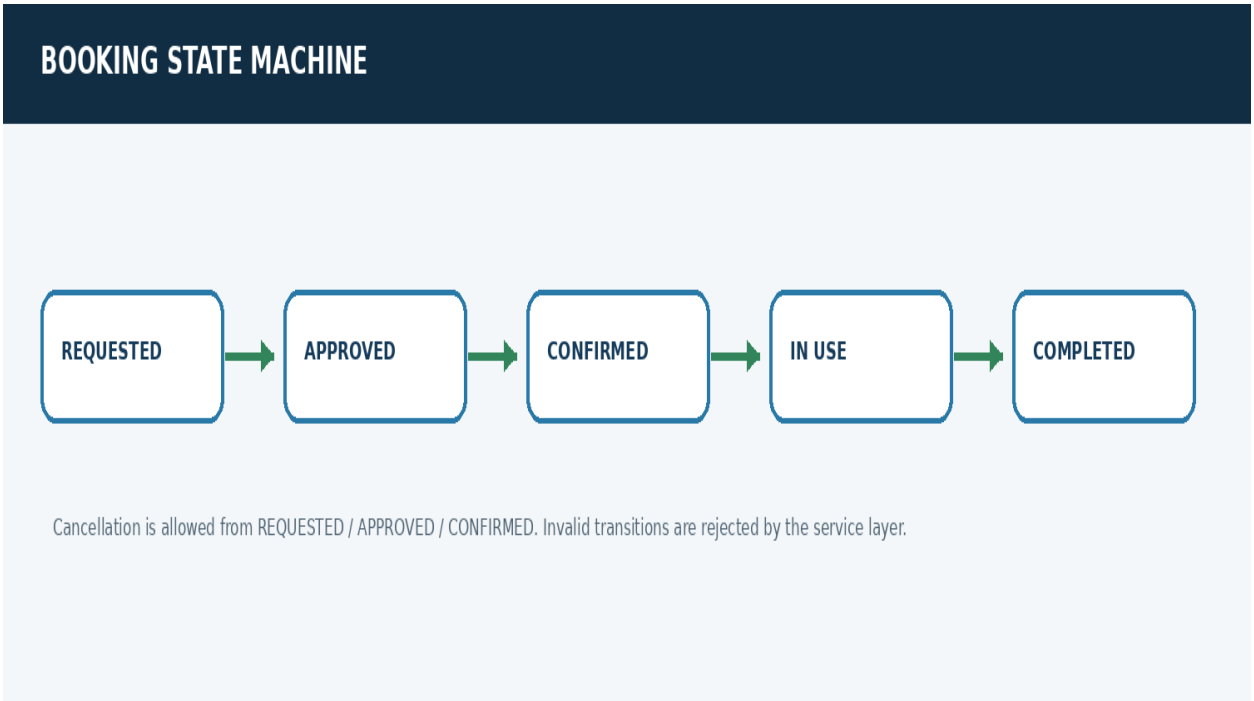


Figure 2. Booking lifecycle and cancellation guard.

Allowed progression: REQUESTED → APPROVED → CONFIRMED → IN_USE → COMPLETED. Cancellation is allowed from REQUESTED, APPROVED and CONFIRMED. Any other transition is rejected. Ticket progression is OPEN → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED.

2.4 Consistency and Duplicate Prevention

Booking overlap is checked on the backend. The service locks the facility row inside a database transaction before checking active overlapping intervals and creating the booking.

Booking state changes use row-level locking and a defined transition map; each accepted transition increments a version and writes an audit event.

Sensor event IDs are unique in PostgreSQL. Replaying an event with the same identity cannot silently create a second event.

Role restrictions are enforced in API permissions and service functions. Frontend navigation is not treated as a security boundary.

An external calendar/issue tracker integration is not implemented in this release. The local database remains authoritative; an adapter and retry queue are future work.

Q3 – Develop the Working Full-Stack Workflow (12 Marks)

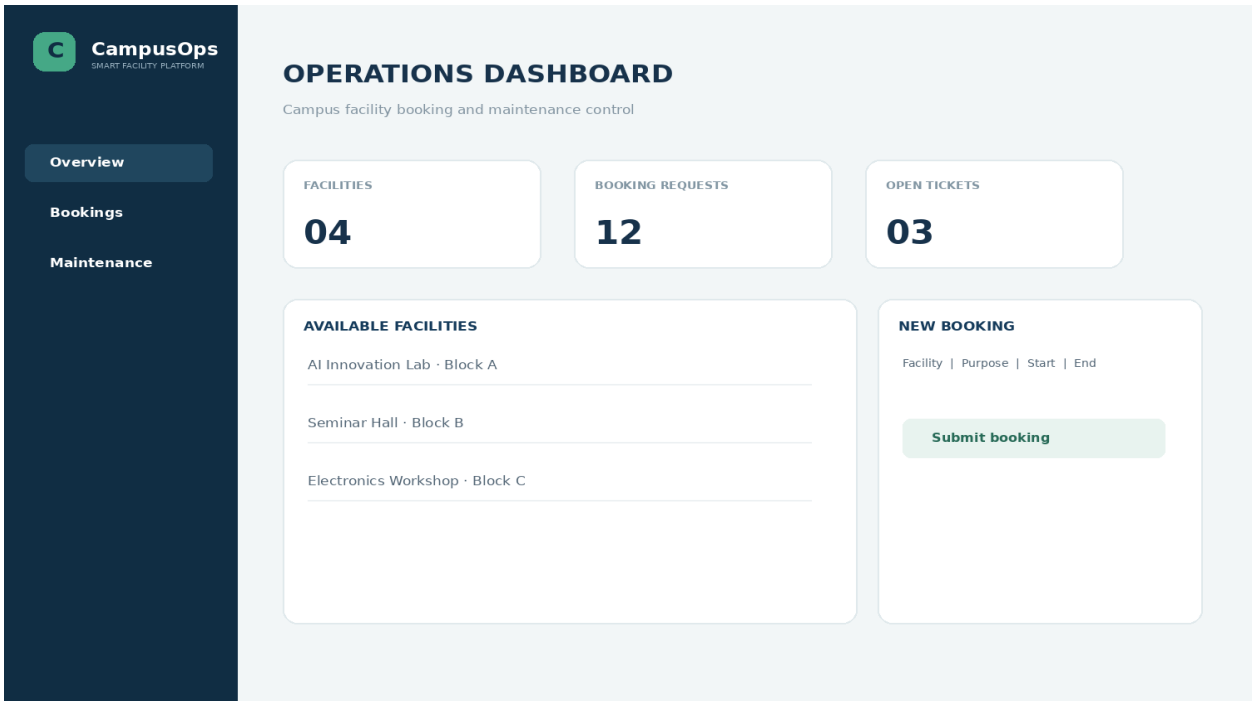


Figure 3. Dashboard design preview generated from the implemented frontend layout; it is not a browser capture.

3.1 Implemented Modules

Module	Frontend	Backend behaviour
Facility catalogue	Lists active facilities and capacity.	GET /api/facilities/; read-only facility view.
Booking	Booking form, recent bookings and transition actions.	Create with overlap validation; guarded approval/confirmation/cancellation.
Maintenance	Ticket form, priority and lifecycle actions.	Create ticket; technician/manager transitions and audit.
Sensor intake	API-first module in this release.	Unique event ID; temperature threshold >=45 creates HIGH ticket.
Audit	Audit endpoint reserved for manager/admin.	Stores actor, entity, action, detail and timestamp.

3.2 End-to-End Demonstration Script

Start the Docker services and wait until PostgreSQL is healthy.

Run Django migrations and the seed_demo command. This creates local demo users and four sample facilities.

Open the frontend at http://localhost:8080 and sign in using the development student account.

Browse the facility catalogue, submit a booking with a future time window, refresh and verify the record remains visible.

Attempt a second booking for the same facility and overlapping time. The API should reject it and preserve the original booking.

Sign in as the manager account, approve the request and confirm it. The booking state should update and be recorded in the audit table.

Create a maintenance ticket, then use a technician/staff account to move it through assignment, progress, resolution and closure.

Post a sensor event with a unique event ID and temperature value of 48.2. Verify that one HIGH ticket is created; replay the event ID and verify uniqueness rejection.

3.3 Evidence to Capture From Your Runtime

Evidence ID	Capture	What it proves
E01	Browser dashboard with facility catalogue	Frontend renders live API data.
E02	Booking request and refreshed booking list	Create/read persistence.
E03	Overlapping booking error	Backend conflict validation.
E04	Manager approval and state change	Role restriction and state transition.
E05	Ticket queue before and after resolution	Maintenance lifecycle.
E06	Sensor event request and generated ticket	Threshold rule and sensor integration.

Do not use the design-preview image as proof of execution. Replace it with actual browser screenshots from your running app before final submission.

Q4 – Establish and Demonstrate Version Control Workflow (6 Marks)

4.1 Repository Structure

smart-campus-platform/ backend/ (Django project and app), frontend/ (Vue application), docs/ (API contracts, test plan and figures), docker-compose.yml, .env.example, README.md and .gitignore.

4.2 Branch and Review Plan

Branch	Change	Review evidence
main	Stable integration branch.	Only reviewed and tested merges; release tag v1.0.0.
feature/booking-overlap	Overlap prevention and booking tests.	Review backend service and negative-case tests.
feature/ticket-sla	Ticket workflow and SLA fields.	Review status transitions and role checks.
fix/duplicate-sensor-event	Unique sensor event handling.	Review replay/duplicate behaviour.

4.3 Commands to Execute

The repository includes a Git workflow guide. Run the following commands in the project folder and capture the actual output:

```
git init
git add .
git commit -m "chore: scaffold smart campus platform"
git switch -c feature/booking-overlap
git commit -am "feat: reject overlapping facility bookings"
git switch main
git switch -c feature/ticket-sla
git commit -am "feat: add maintenance ticket workflow"
git switch main
git merge --no-ff feature/booking-overlap
git merge --no-ff feature/ticket-sla
git tag -a v1.0.0 -m "Assignment 3 release"
git log --oneline --graph --decorate --all
```

Conflict demonstration: create two branches that edit the same booking policy line, merge one branch and then merge the second. Resolve the real conflict, run tests and record the actual Git output. Do not invent commit IDs or claim a merge occurred without performing it.

Q5 – Create and Validate Reproducible Container Deployment (8 Marks)

5.1 Container Design

Service	Image/build	Port / storage	Responsibility
frontend	Vue build stage + Nginx runtime	8080:80	Static frontend assets.
backend	Python 3.12 slim + Django/Gunicorn	8000:8000	REST API and migrations.
db	PostgreSQL 16 Alpine	Internal 5432; named volume postgres_data	Persistent application data.

5.2 Deployment Commands

```
cp .env.example .env # replace local secrets
docker compose up --build -d
docker compose ps
docker compose exec backend python manage.py makemigrations campus
docker compose exec backend python manage.py migrate
docker compose exec backend python manage.py seed_demo
curl http://localhost:8000/api/health/
docker compose logs --tail=100 backend frontend db
```

5.3 Persistence and Restart Test

Create a booking and record its ID and status.

Restart only the backend: docker compose restart backend.

Sign in again and retrieve the same booking ID. A successful API health check alone does not prove persistence.

Restart the database container only after it is healthy and use the named volume. Verify the same booking and ticket records still exist.

Rebuild the frontend/backend containers from source and repeat the end-to-end journey.

The Compose file defines a PostgreSQL named volume, but persistence is only demonstrated after the restart test is actually run. No deployment result is claimed in this report.

Q6 – Adapt the Application for Concurrent Access and Deployment Changes (8 Marks)

6.1 Impact of the Changed Scenario

Change	Impact	Adaptation
Six campuses	Facility identifiers and access boundaries must include campus.	Add campus/region field and scope manager/technician access by campus.
60 → 300 peak users	Concurrent booking requests and API/database load increase.	Stateless API replicas, DB connection pooling, indexes and measured load test.
Two simultaneous same-slot reservations	Race can permit duplicate bookings if check and insert are separate.	Lock facility row in transaction; test simultaneous requests against PostgreSQL.
Two technicians update same ticket	Last-write-wins can hide one update.	Use version field / If-Match expected version and return conflict on stale update.
One backend instance restarts	In-flight request may fail; persistent data must survive.	Multiple backend replicas later; PostgreSQL volume; retry safe GET/command with idempotency key.

6.2 Revised Design

Add Campus entity and campus_id foreign keys to Facility, Booking and MaintenanceTicket; enforce campus scope in querysets and permissions.

Keep PostgreSQL as the source of truth. Add indexes on facility/start/end, status, campus and ticket SLA fields.

Add an expected_version field to mutation contracts. Reject stale updates with HTTP 409 rather than silently overwriting another technician's change.

Run two backend instances behind a reverse proxy only after the database connection budget and health checks are configured.

Use bounded retries for transient failures. Do not blindly retry non-idempotent booking creation without a client-generated idempotency key.

Use PostgreSQL transactions and row locks for reservation correctness. Validate actual isolation behaviour under concurrent clients.

6.3 Validation Matrix

Case	Procedure	Expected / reporting requirement
C01 Overlap race	Send two simultaneous booking requests for same facility/time.	Exactly one active booking is created; second receives conflict. Record HTTP status and DB rows.
C02 Stale technician update	Two clients read version N; first updates; second submits version N.	Second receives 409; first update remains; audit captures accepted change.
C03 Retry	Repeat request after simulated timeout with same idempotency key.	One logical booking/ticket only.
C04 Backend restart	Create records, restart one backend instance and retrieve records.	Existing records remain; report interruption and recovery time.
C05 300-user target	Increase load gradually (e.g. 10, 25, 50, 100, 200, 300).	Report throughput, p95 latency, error rate and resource use; do not claim target without measurement.

Current implementation includes transactional facility-row locking for booking creation and a version field incremented by transitions. A full two-instance optimistic concurrency check, campus scoping and 300-user load test are identified as required extensions, not completed features.

Q7 – Individual Technical Reflection (4 Marks)

The most influential integration decision is keeping booking conflict prevention in the Django service and PostgreSQL transaction rather than relying on the Vue calendar. A frontend-only check can become stale when two users submit at the same time. The service locks the facility row, checks overlapping active bookings and creates the request in one transaction.

A key defect risk is a duplicate or overlapping booking caused by a check-then-insert race. The design addresses it with a transaction and facility-row lock. The required next verification is a simultaneous request test against PostgreSQL; sequential tests alone do not prove race safety.

Git branches and review records make parallel contributions visible and allow conflict resolution before release. Docker Compose pins service boundaries and keeps PostgreSQL data in a named volume, improving reproducibility. Neither tool guarantees correctness: secrets, migration order, health checks, persistent-volume verification and real merge evidence still need review.

The main remaining limitation is that the assignment's six-campus, 300-user scenario has not been measured. Before wider deployment, campus-scoped authorization, optimistic version checks, automated concurrency tests, monitoring, backup/restore drills, secure production authentication and load testing should be completed.

Appendix A – Repository and Execution Evidence

The accompanying ZIP contains the full source tree, Docker Compose configuration, API contract, test plan, Git workflow guide and architecture images. These assets are source artefacts, not proof that Docker or GitHub has been run on the student's machine.

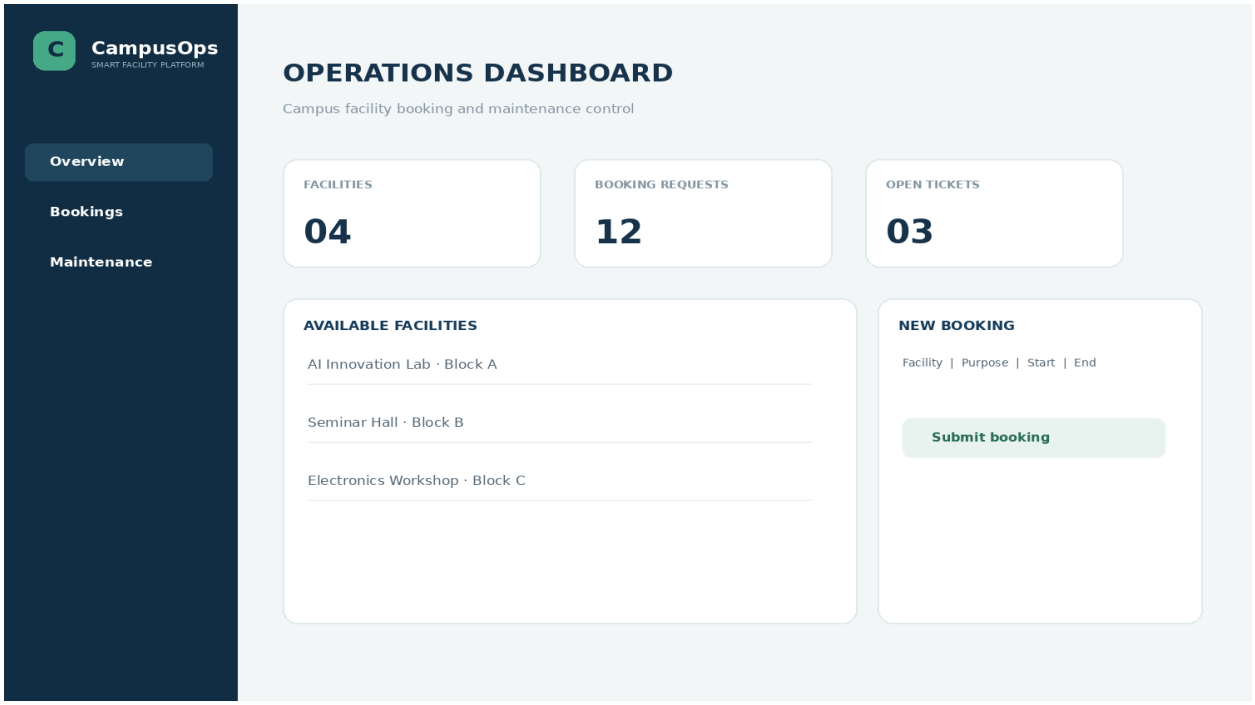


Figure A1. UI design preview only; replace with actual browser capture after runtime.

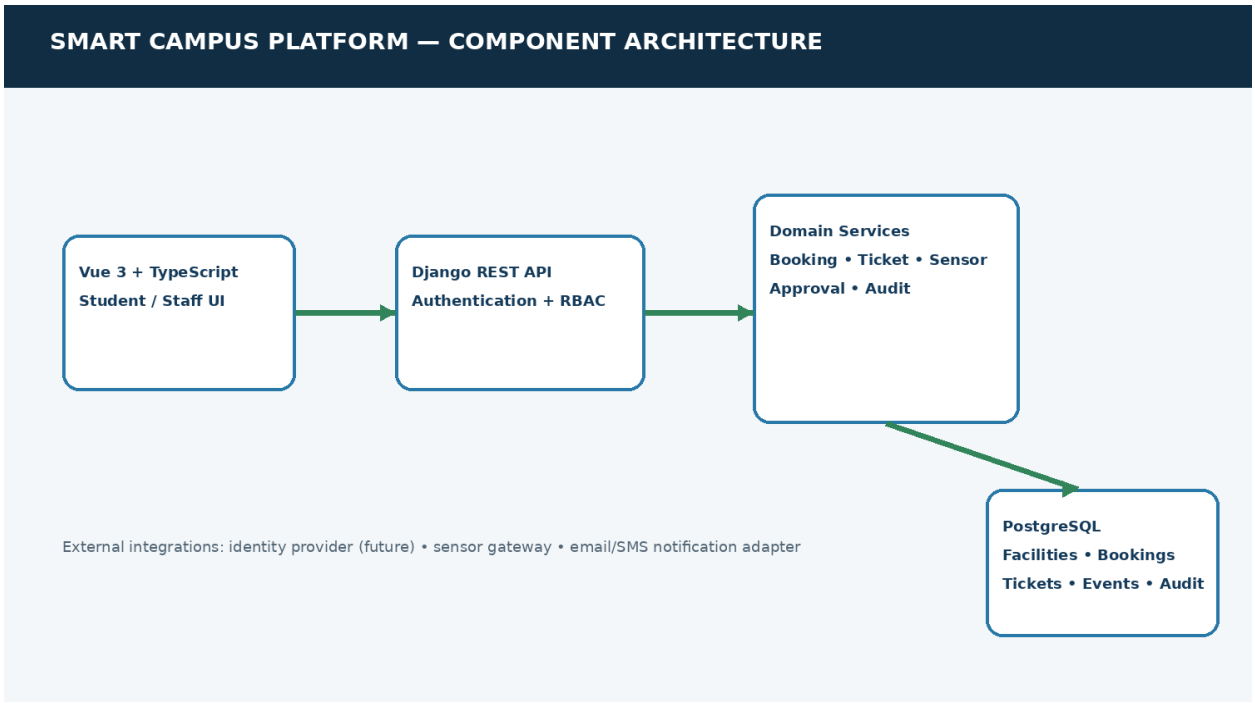


Figure A2. Architecture diagram.

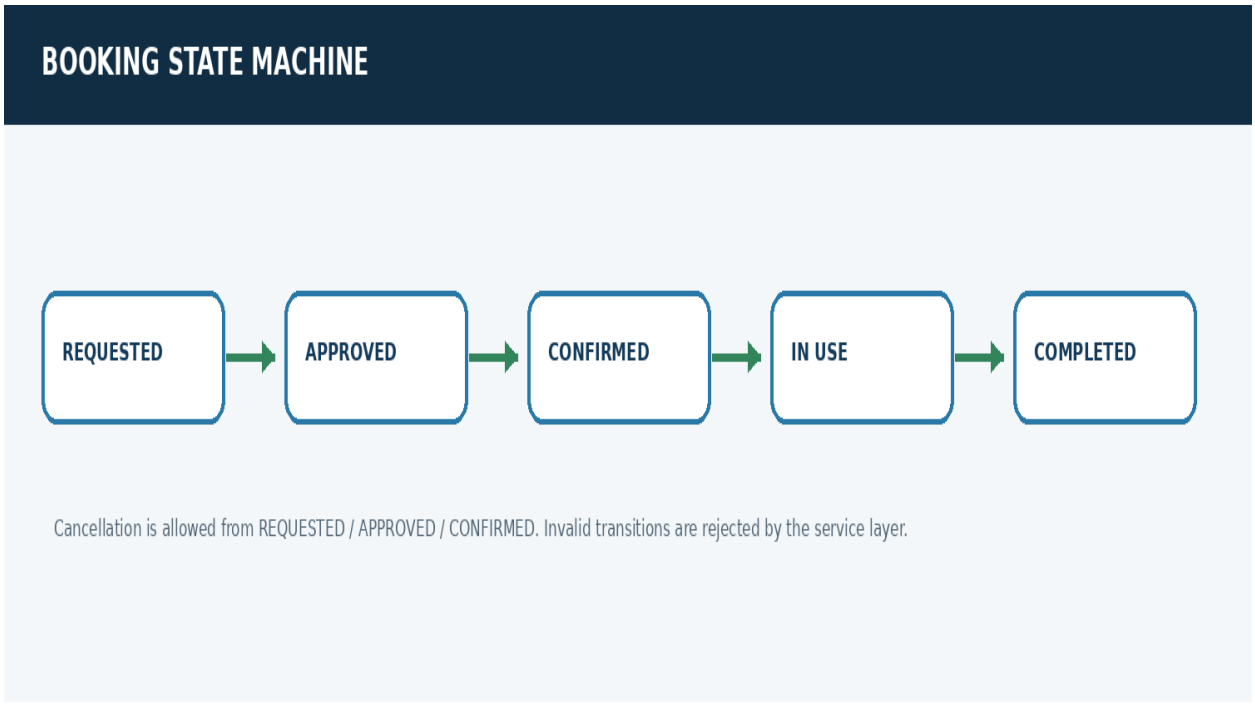


Figure A3. Booking workflow diagram.

Appendix B – Limitations and Submission Checklist

Run Docker Compose on the student's laptop and capture container status and logs.

Run migrations, seed demo data and capture the API health response.

Complete the booking, overlap rejection, approval, ticket and sensor-event journey.

Run Django tests and record actual test output.

Create the GitHub repository, branches, commits, merge review and tag; capture genuine evidence.

Restart backend and database containers and verify persisted booking/ticket IDs.

Run the bounded concurrency experiment and report measured results; do not claim 300-user support without data.

Replace the UI design preview with screenshots from the running application.

References

Django documentation: https://docs.djangoproject.com/

Django REST Framework documentation: https://www.django-rest-framework.org/

Vue.js documentation: https://vuejs.org/guide/

PostgreSQL documentation: https://www.postgresql.org/docs/

Docker Compose documentation: https://docs.docker.com/compose/

Git documentation: https://git-scm.com/docs/

CSA10 Software Engineering Assignment 3 question paper and assessment guidance supplied in the uploaded PDF.